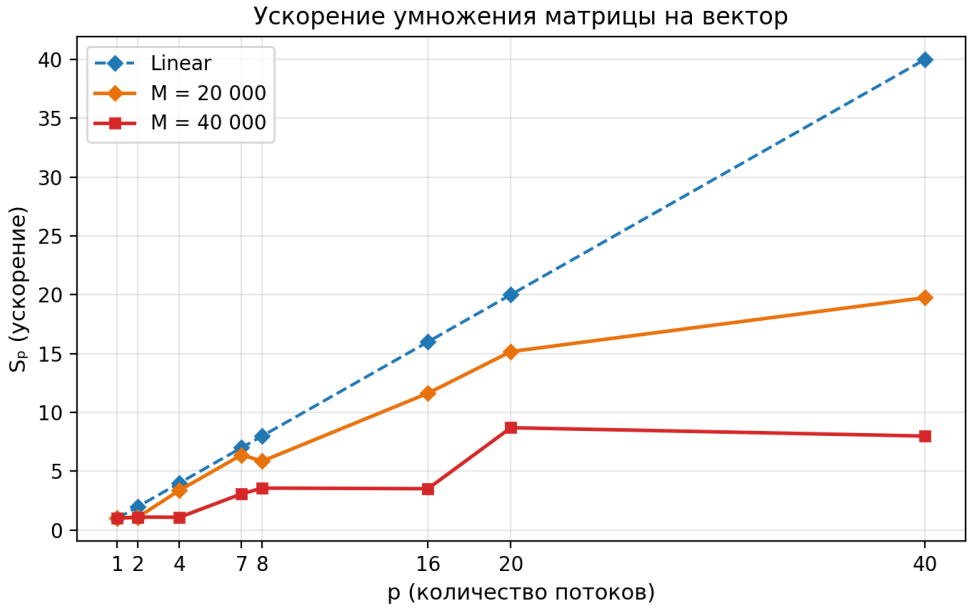
Задача 1

Параметр	Значение
Сервер	HP ProLiant XL270d Gen10
Процессор	2 x Intel Xeon Gold 6248 @ 2.50 GHz (Turbo до 3.90 GHz)
Архитектура	x86_64, Cascade Lake
Сокетов / Ядер / Потоков	2 / 20 на сокет / 2 на ядро (80 логических CPU)
Кэш L1d / L2 / L3	32 KiB / 1 MiB / 27.5 MiB на сокет
NUMA-узлов	2 (Node 0: 385 636 MiB, Node 1: 387 008 MiB)
ОЗУ	754 GiB DDR4
ОС	Ubuntu 22.04.5 LTS

M = N	Количество потоков														
	2			4		7		8		16		20		40	
	T_1	T_2	S_2	T_4	S_4	T_7	S_7	T_8	S_8	T_{16}	S_{16}	T_{20}	S_{20}	T_{40}	S_{40}
20000 (~3 GiB)	0.61	0.58	**1.10**	0.19	**3.38**	0.10	**6.39**	0.11	**5.86**	0.05	**11.63**	0.04	**15.17**	0.03	**19.75**
40000 (~12 GiB)	2.13	2.10	**1.11**	2.12	**1.09**	0.70	**3.08**	0.63	**3.57**	0.73	**3.52**	0.29	**8.70**	0.31	**7.99**



Размер 20 000 x 20 000 (~3 GiB).

Масштабируемость хорошая. Ускорение растёт почти линейно до 7 потоков (S7 = 6.39, E = 0.91), затем продолжает расти, но эффективность падает. На 20 потоках достигнуто S20 = 15.17. При переходе к 40 потокам (Hyper-Threading + 2 сокета) ускорение S40 = 19.75, но эффективность упала до 0.49 из-за Hyper-Threading и NUMA-эффектов.

Размер 40 000 x 40 000 (~12 GiB).

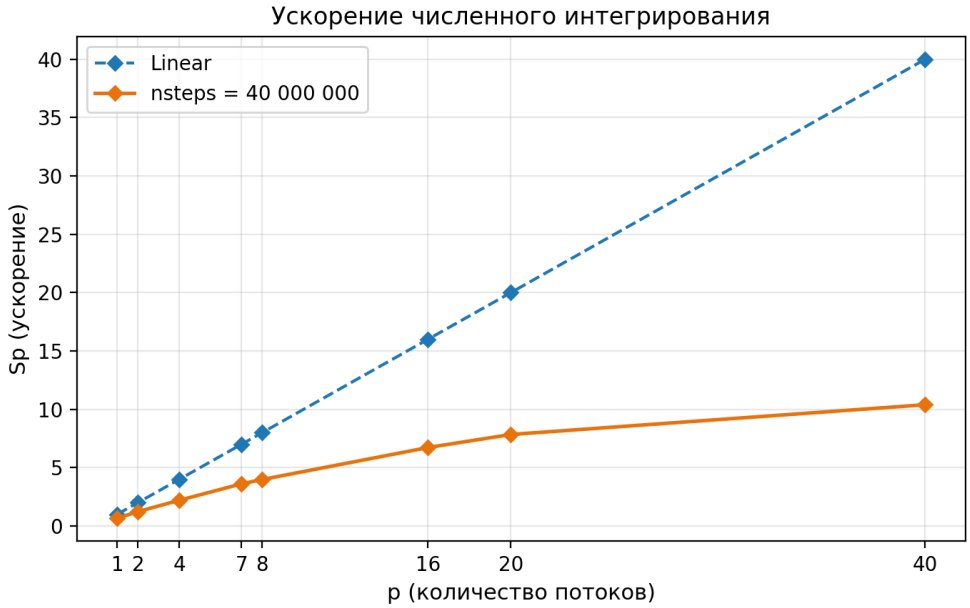
Масштабируемость значительно хуже. Матрица занимает ~12 GiB, что превышает объём памяти одного NUMA-узла. При 2-4 потоках ускорение минимально (S2 = 1.11, S4 = 1.09) из-за NUMA-эффектов. Максимум S20 = 8.70. При 40 потоках наблюдается деградация (S40 = 7.99) из-за конкуренции за пропускную способность шины памяти.

Общий вывод.

Умножение матрицы на вектор - memory-bound операция. Для матриц, помещающихся в пределах одного NUMA-узла, масштабируемость хорошая. Для больших матриц масштабируемость ограничена пропускной способностью межузловых соединений. Рекомендуется привязка потоков к ядрам (thread pinning) и NUMA-aware распределение памяти.

Задача 2

Nsteps a = -4.0 b = 4.0	Количество потоков														
	2			4		7		8		16		20		40	
	T_1	T_2	S_2	T_4	S_4	T_7	S_7	T_8	S_8	T_{16}	S_{16}	T_{20}	S_{20}	T_{40}	S_{40}
40 000 000	0.29	0.23	**1.23**	0.12	**2.20**	0.07	**3.61**	0.07	**3.98**	0.04	**6.72**	0.04	**7.84**	0.03	**10.39**



Численное интегрирование - задача типа embarrassingly parallel: каждая точка вычисляется независимо, нет обращений к разделяемой памяти (кроме финальной atomic-операции).

Тем не менее наблюдается отклонение от линейного ускорения: S8 = 3.98 (идеал = 8), S40 = 10.39 (идеал = 40). Причины:

1) Малый объём вычислений: T1 = 0.289 сек. Накладные расходы на создание потоков и atomic-синхронизацию становятся заметной долей.

2) Turbo Boost: при нагрузке на 1 ядро частота до 3.9 ГГц, при нагрузке на все ядра - ~3.0 ГГц, что снижает speedup на 20-30%.

3) NUMA-эффекты при >20 потоках: кэш-когерентность при atomic добавляет задержки.

4) Hyper-Threading: потоки делят физические ядра, что не удваивает пропускную способность для compute-bound задач.

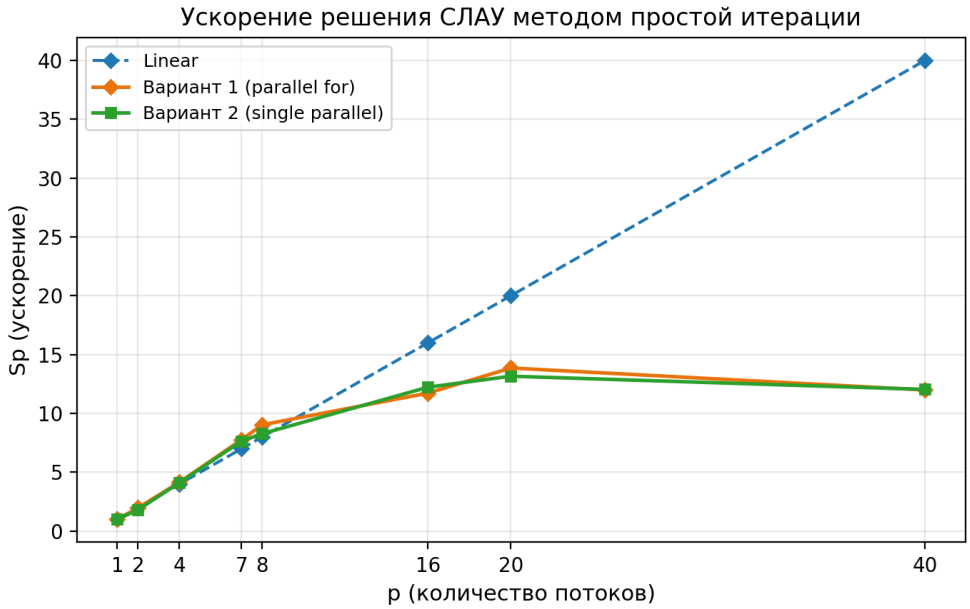
Общий вывод.

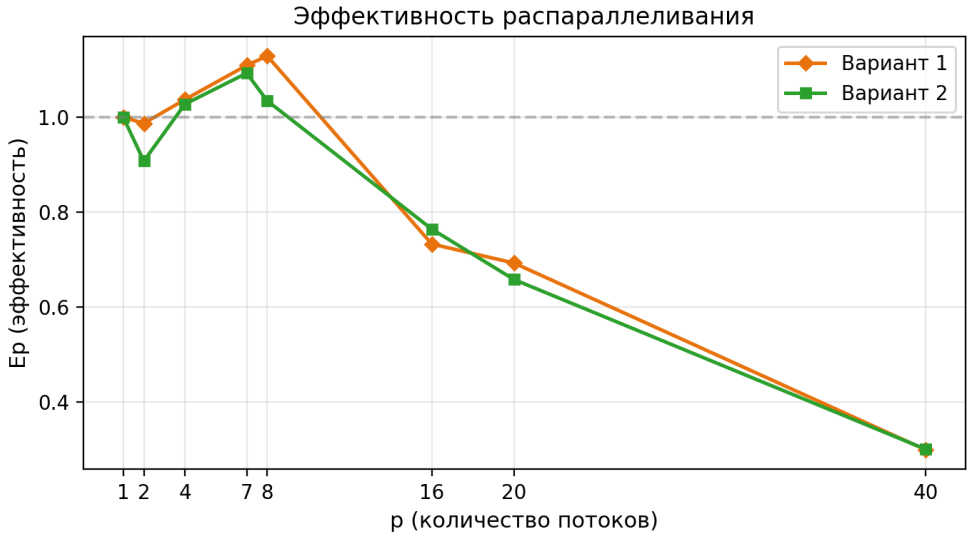
Масштабируемость ограничена малым объёмом работы и накладными расходами. Для улучшения рекомендуется увеличить nsteps и/или использовать reduction вместо atomic. Тем не менее, ускорение в 10.39 раз на 40 потоках демонстрирует практическую пользу распараллеливания.

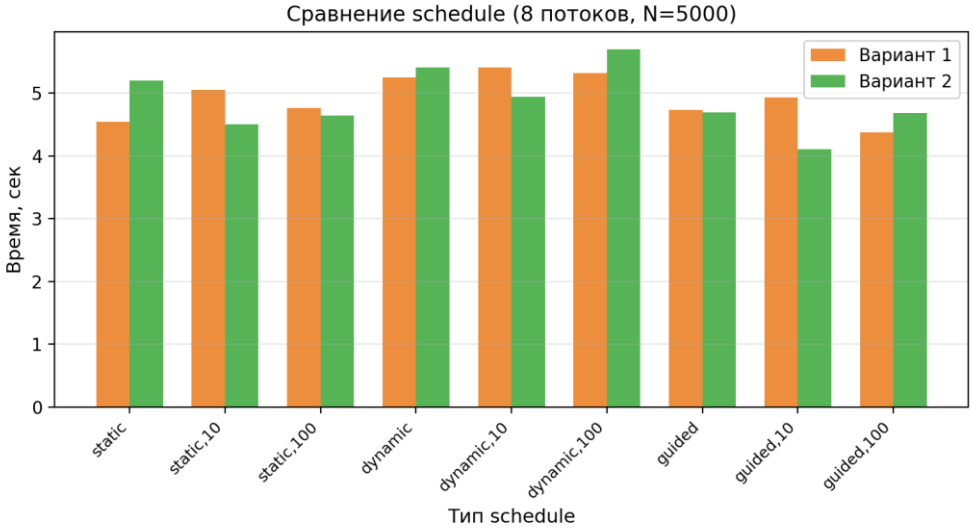
Задача 3

Задача 3															
N = 1500 Solved for 4 iterations tau = 6.67e-4	Количество потоков														
	2			4		7		8		16		20		40	
	T_1	T_2	S_2	T_4	S_4	T_7	S_7	T_8	S_8	T_{16}	S_{16}	T_{20}	S_{20}	T_{40}	S_{40}
(#pragma omp parallel for)	0.0050	0.0042	**1.19**	0.0024	**2.12**	0.0018	**2.71**	0.0017	**2.92**	0.0021	**2.39**	0.0022	**2.24**	0.0041	**1.21**
(#pragma omp parallel)	0.0046	0.0045	**1.02**	0.0026	**1.75**	0.0019	**2.48**	0.0019	**2.47**	0.0022	**2.10**	0.0024	**1.94**	0.0040	**1.14**

schedule	V1, сек	V2, сек
static	0.0016	0.0019
static,10	0.0017	0.0015
static,100	0.0017	0.0017
dynamic	0.0027	0.0027
dynamic,10	0.0019	0.0019
dynamic,100	0.0017	0.0020
guided	0.0019	0.0018
guided,10	0.0018	0.0022
guided,100	0.0022	0.0023







1. Оба варианта демонстрируют хорошую масштабируемость до 20 потоков (одного физического сокета). Максимальное ускорение: V1: S20 = 13.86, V2: S20 = 13.16.

2. При 40 потоках наблюдается деградация: S40 < S20 для обоих вариантов. Причины: Hyper-Threading не увеличивает пропускную способность для memory-bound задач, конкуренция за шину памяти, NUMA-эффекты.

3. Сверхлинейное ускорение (E > 1.0) при 4-8 потоках объясняется кэш-эффектами: при распределении строк матрицы по большему числу ядер рабочее подмножество данных лучше помещается в кэш.

4. Разница между вариантами 1 и 2 незначительна (0-9%). V1 чуть быстрее при малом числе потоков, V2 - при 16+.

5. Оптимальные параметры schedule: static или guided с chunk ~100. dynamic не рекомендуется - нагрузка равномерна, а накладные расходы значительны.

Вариант 2 предпочтительнее в общем случае - лучше масштабируется при большом числе потоков, нет накладных расходов на многократное создание thread pool.